[ตราสัญลักษณ์สถาบัน]

ปริญญานิพนธ์

การพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล

(Digital Signage Management System)

โดย

[ชื่อ-สกุลนักศึกษา] [รหัสนักศึกษา]

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตาม

หลักสูตร[ปริญญา] สาขาวิชา[สาขาวิชา]

[คณะ / วิทยาลัย]

[มหาวิทยาลัย]

ปีการศึกษา [25XX]

ใบรับรองปริญญานิพนธ์

[คณะ / วิทยาลัย] [มหาวิทยาลัย]

หัวข้อ : การพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล

โดย : [ชื่อ-สกุลนักศึกษา] [รหัสนักศึกษา]

.....

([อาจารย์ที่ปรึกษา])

.....

([กรรมการ])

.....

([คณบดี / หัวหน้าสาขา])

บทคัดย่อ

ระบบจัดการจอประชาสัมพันธ์ดิจิทัล (Digital Signage Management System) พัฒนาขึ้นเพื่อแก้ปัญหาการบริหารจัดการจอประชาสัมพันธ์หลายจุดในองค์กรที่เดิมต้องอัปเดตเนื้อหาและควบคุมจอด้วยมือทีละเครื่อง โดยรวมการจัดการเนื้อหา การจัดผังการแสดงผล การกำหนดตารางเวลาออกอากาศ การควบคุมจอโทรทัศน์และจอ LED ระยะไกลผ่านโปรโตคอล HDMI-CEC และ RS-232 ตลอดจนการประกาศเหตุฉุกเฉิน ไว้ในระบบกลางเพียงระบบเดียว

ระบบพัฒนาด้วยสถาปัตยกรรมแบบบริการหลายส่วน (FastAPI, PostgreSQL, Redis, MinIO และ Next.js) ทำงานในรูปแบบคอนเทนเนอร์ด้วย Docker มีระบบสิทธิ์ผู้ใช้แบบ RBAC กระบวนการอนุมัติเนื้อหา การบันทึกประวัติการใช้งาน และการเฝ้าระวังสถานะอุปกรณ์ ส่วนแสดงผลรองรับการทำงานแบบ offline-friendly ผ่านไฟล์ manifest ผลการพัฒนาได้ระบบที่ใช้งานได้ครบวงจรพร้อมการทดสอบอัตโนมัติประมาณ 133 กรณี

คำสำคัญ: ป้ายประชาสัมพันธ์ดิจิทัล, การจัดการเนื้อหา, การควบคุมทีวีระยะไกล, HDMI-CEC, RS-232, FastAPI

ABSTRACT

The Digital Signage Management System was developed to address the challenge of managing multiple public-display screens within an organization, which previously required manual content updates and per-device screen control. The system consolidates content management, layout composition, broadcast scheduling, remote control of televisions and LED displays via HDMI-CEC and RS-232, and emergency announcements into a single centralized platform.

Built on a multi-service architecture (FastAPI, PostgreSQL, Redis, MinIO, and Next.js) running in Docker containers, the system provides role-based access control (RBAC), a content approval workflow, audit logging, and device monitoring. The player supports offline-friendly operation through manifest files. The result is a fully functional system accompanied by approximately 133 automated test cases.

Keywords: Digital Signage, Content Management, Remote TV Control, HDMI-CEC, RS-232, FastAPI

กิตติกรรมประกาศ

ปริญญานิพนธ์ฉบับนี้สำเร็จลุล่วงได้ด้วยความกรุณาและการสนับสนุนจาก [สถานประกอบการ] และ [อาจารย์ที่ปรึกษา] ผู้จัดทำขอขอบพระคุณเป็นอย่างสูงมา ณ โอกาสนี้

[ข้อความขอบคุณเพิ่มเติม — เติมเอง]

สารบัญ

หน้า

บทคัดย่อภาษาไทย **3**

บทคัดย่อภาษาอังกฤษ **4**

กิตติกรรมประกาศ **5**

บทที่ 1 บทนำ **8**

1.1 ความเป็นมาและความสำคัญของโครงงาน 8

1.2 วัตถุประสงค์ของโครงงาน 8

1.3 ขอบเขตของโครงงาน 9

1.3.1 ขอบเขตด้านปริมาณการใช้งาน 9

1.3.2 แพลตฟอร์มของจอแสดงผลที่รองรับ 9

1.3.3 การควบคุมจอโทรทัศน์และอุปกรณ์แสดงผล 9

1.3.4 ชนิดของเนื้อหาที่รองรับ 9

1.3.5 การติดตั้งและการนำไปใช้งาน 9

1.3.6 ข้อจำกัดของโครงงาน 9

1.4 ประโยชน์ที่คาดว่าจะได้รับ 10

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง **11**

2.1 ระบบป้ายประชาสัมพันธ์ดิจิทัล (Digital Signage) 11

2.2 FastAPI และภาษา Python 11

2.3 PostgreSQL, SQLAlchemy และ Alembic 12

2.4 Next.js, TypeScript และ Tailwind CSS 12

2.5 Redis และ Arq 12

2.6 MinIO 13

2.7 Docker และการทำคอนเทนเนอร์ (Containerization) 13

2.8 HDMI-CEC และ RS-232 13

2.9 ความปลอดภัยของระบบ (Security) 14

บทที่ 3 การวิเคราะห์และออกแบบระบบ **15**

3.1 การวิเคราะห์ความต้องการของระบบ 15

3.2 เครื่องมือที่ใช้ในการพัฒนา 15

3.2.1 ฮาร์ดแวร์ (Hardware) 15

3.2.2 ซอฟต์แวร์ (Software) 15

3.3 การออกแบบระบบ 16

3.3.1 สถาปัตยกรรมระบบ (System Architecture) 16

3.3.2 การออกแบบฐานข้อมูล (ER Diagram) 16

3.3.3 แนวคิดลำดับชั้นเนื้อหาแบบ 5 ระดับ (5-Tier Content Hierarchy) 16

3.3.4 การออกแบบส่วนติดต่อโปรแกรม (API) 16

3.4 วิธีการพัฒนา 16

3.5 การออกแบบการทดสอบ 17

บทที่ 4 ผลการดำเนินงาน **18**

4.1 ภาพรวมระบบที่พัฒนา 18

4.2 โมดูลและความสามารถหลัก 18

4.3 หน้าจอส่วนจัดการสำหรับผู้ดูแล (Admin UI) 19

4.4 ส่วนแสดงผล (Player) 19

บทที่ 5 สรุปผลและข้อเสนอแนะ **20**

5.1 สรุปผลการดำเนินงาน 20

5.2 ปัญหาที่พบและแนวทางแก้ไข 20

5.3 ข้อเสนอแนะและการพัฒนาต่อยอด 20

5.4 สิ่งที่ได้เรียนรู้จากการฝึกงาน 21

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของโครงงาน

ในปัจจุบัน องค์กรขนาดใหญ่ เช่น โรงพยาบาล สถานศึกษา และหน่วยงานราชการ นิยมใช้จอแสดงผลเพื่อประชาสัมพันธ์ข้อมูลข่าวสารแก่ผู้มาใช้บริการตามจุดต่าง ๆ ภายในอาคาร ระบบลักษณะนี้เรียกว่า "ระบบป้ายประชาสัมพันธ์ดิจิทัล" (Digital Signage) ซึ่งช่วยให้การสื่อสารข้อมูลทำได้รวดเร็วและปรับเปลี่ยนได้ตลอดเวลา อย่างไรก็ตาม เมื่อจำนวนจอเพิ่มมากขึ้นและกระจายอยู่หลายจุด การบริหารจัดการเนื้อหาและการควบคุมจอแต่ละเครื่องด้วยวิธีเดิม เช่น การนำไฟล์ใส่อุปกรณ์ USB ไปเสียบที่จอทีละเครื่อง หรือการเดินไปเปิด-ปิดจอด้วยมือ ก่อให้เกิดปัญหาหลายประการ ได้แก่ ความล่าช้าในการอัปเดตเนื้อหา ความไม่สอดคล้องของข้อมูลระหว่างจอ ความยากในการกำหนดตารางการแสดงผล และการขาดช่องทางในการประกาศข้อมูลเร่งด่วนหรือเหตุฉุกเฉินได้ทันท่วงที

จากปัญหาดังกล่าว ผู้จัดทำจึงได้พัฒนา "ระบบจัดการจอประชาสัมพันธ์ดิจิทัล (Digital Signage Management System)" ขึ้น เพื่อรวมการจัดการเนื้อหา การจัดผังการแสดงผล (layout) การกำหนดตารางเวลาออกอากาศ (schedule) การควบคุมจอโทรทัศน์ระยะไกล ตลอดจนการประกาศเหตุฉุกเฉิน ไว้ในระบบกลางเพียงระบบเดียว ผู้ดูแลสามารถสั่งงานผ่านเว็บแอปพลิเคชันจากศูนย์กลางได้ โดยไม่ต้องเดินไปจัดการที่หน้าจอแต่ละเครื่อง ช่วยลดทั้งเวลาและความผิดพลาดในการดำเนินงาน

ระบบออกแบบให้ทำงานภายในเครือข่ายภายในองค์กร (LAN) รองรับการเล่นเนื้อหาแบบ offline-friendly ผ่านไฟล์ manifest เพื่อให้จอยังแสดงผลต่อได้แม้เครือข่ายขัดข้องชั่วคราว มีระบบสิทธิ์ผู้ใช้ (RBAC) และกระบวนการอนุมัติเนื้อหาเพื่อความปลอดภัยและความถูกต้องของข้อมูล รวมถึงการบันทึกประวัติการใช้งาน (audit log) และการเฝ้าระวังสถานะอุปกรณ์ นอกจากนี้ยังพัฒนาด้วยเทคโนโลยี Open Source ทั้งหมด และติดตั้งด้วย Docker เพื่อให้ปรับใช้และดูแลรักษาได้ง่าย ลดต้นทุนค่าลิขสิทธิ์ซอฟต์แวร์

1.2 วัตถุประสงค์ของโครงงาน

1. เพื่อพัฒนาระบบจัดการเนื้อหาและตารางการแสดงผลของจอประชาสัมพันธ์จากศูนย์กลาง

2. เพื่อรองรับการควบคุมจอโทรทัศน์ระยะไกล (เปิด/ปิด/เปลี่ยนช่องสัญญาณเข้า) ผ่านโปรโตคอล HDMI-CEC และ RS-232 ได้หลายยี่ห้อ
3. เพื่อจัดให้มีระบบสิทธิ์ผู้ใช้ (RBAC) และกระบวนการอนุมัติเนื้อหาและตารางก่อนเผยแพร่
4. เพื่อรองรับการประกาศเหตุฉุกเฉินที่สามารถแทรก (override) ตารางการแสดงผลปกติได้ทันที
5. เพื่อเฝ้าระวังสถานะของอุปกรณ์และบันทึกประวัติการกระทำ (audit log) ทุกรายการ
6. เพื่อพัฒนาระบบด้วยเทคโนโลยี Open Source และสามารถติดตั้งใช้งานด้วย Docker ได้

1.3 ขอบเขตของโครงงาน

1.3.1 ขอบเขตด้านปริมาณการใช้งาน

รองรับการบริหารจัดการจอประชาสัมพันธ์ระดับ 10–20 จุด (ขนาด Proof of Concept) และสามารถขยายจำนวนได้ผ่านการจัดกลุ่มจอ (player group)

1.3.2 แพลตฟอร์มของจอแสดงผลที่รองรับ

รองรับ Android TV, แท็บเล็ต Android, Raspberry Pi (โหมด kiosk), เครื่องคอมพิวเตอร์ Windows และเว็บเบราว์เซอร์ (PWA)

1.3.3 การควบคุมจอโทรทัศน์และอุปกรณ์แสดงผล

รองรับการสั่งงานทีวีผ่าน HDMI-CEC และ RS-232 ได้หลายยี่ห้อ (เช่น Samsung MDC, LG, Sony) รวมถึงการควบคุมจอ LED ผ่านโมดูลควบคุม LED

1.3.4 ชนิดของเนื้อหาที่รองรับ

รองรับเนื้อหาประเภทรูปภาพ (image) วิดีโอ (video) หน้าเว็บ (web) สตรีม (stream) และข้อความ (text)

1.3.5 การติดตั้งและการนำไปใช้งาน

ออกแบบให้ติดตั้งแบบ on-premise บนระบบปฏิบัติการ Linux (Ubuntu 22.04) ด้วย Docker Compose

1.3.6 ข้อจำกัดของโครงงาน

เป็นการพัฒนาระบบใหม่ (greenfield) จึงไม่รวมการย้ายข้อมูลจากระบบเดิม (data migration)

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ลดเวลาและข้อผิดพลาดในการอัปเดตเนื้อหา เนื่องจากบริหารจัดการได้จากศูนย์กลาง
2. ลดภาระการเดินไปควบคุมจอแต่ละเครื่อง ด้วยการสั่งเปิด/ปิด/เปลี่ยนสัญญาณจากระยะไกล
3. เพิ่มความปลอดภัยและความถูกต้องของข้อมูล ด้วยระบบสิทธิ์ผู้ใช้และกระบวนการอนุมัติ
4. สามารถสื่อสารข้อมูลเร่งด่วนหรือเหตุฉุกเฉินไปยังทุกจอได้ทันที
5. ลดต้นทุนค่าลิขสิทธิ์ซอฟต์แวร์ ด้วยการใช้เทคโนโลยี Open Source
6. ได้ระบบต้นแบบที่สามารถนำไปขยายผลและต่อยอดในองค์กรจริงได้

บทที่ 2
ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

ในบทนี้กล่าวถึงทฤษฎีและเทคโนโลยีที่นำมาใช้ในการพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล เพื่อให้เข้าใจหลักการทำงานของเครื่องมือแต่ละส่วนก่อนนำไปสู่การออกแบบและพัฒนาในบทต่อไป

2.1 ระบบป้ายประชาสัมพันธ์ดิจิทัล (Digital Signage)

ระบบป้ายประชาสัมพันธ์ดิจิทัล คือระบบที่ใช้จอแสดงผลอิเล็กทรอนิกส์ในการนำเสนอเนื้อหาดิจิทัล เช่น รูปภาพ วิดีโอ และข้อความ โดยสามารถควบคุมและปรับเปลี่ยนเนื้อหาจากศูนย์กลางผ่านเครือข่าย องค์ประกอบหลักประกอบด้วยส่วนจัดการเนื้อหา (Content Management) ส่วนแสดงผลที่ติดตั้งกับจอ (Player) และช่องทางสื่อสารระหว่างสองส่วนนี้ ทำให้องค์กรสามารถอัปเดตเนื้อหาและกำหนดตารางการแสดงผลล่วงหน้าได้อย่างมีประสิทธิภาพ

[ภาพประกอบ: ระบบ Digital Signage]

รูปที่ 2.1 ระบบ Digital Signage

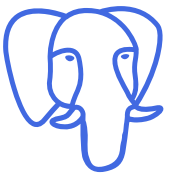
2.2 FastAPI และภาษา Python

FastAPI เป็นเว็บเฟรมเวิร์กภาษา Python สำหรับสร้าง REST API ที่ทำงานแบบ asynchronous มีจุดเด่นด้านประสิทธิภาพสูง การตรวจสอบชนิดข้อมูลด้วย type hints และการสร้างเอกสาร API อัตโนมัติในรูปแบบ OpenAPI (Swagger) ในระบบนี้ใช้ FastAPI 0.115 ร่วมกับ Python 3.12 เป็นแกนหลักของฝั่ง backend สำหรับจัดการตรรกะทางธุรกิจและให้บริการ API แก่ส่วนอื่น ๆ

รูปที่ 2.2 FastAPI

2.3 PostgreSQL, SQLAlchemy และ Alembic

PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (RDBMS) แบบ open source ที่มีความเสถียรและรองรับข้อมูลปริมาณมาก ระบบนี้ใช้ PostgreSQL 16 เป็นฐานข้อมูลหลัก โดยเข้าถึงผ่าน SQLAlchemy 2.x ซึ่งเป็น Object-Relational Mapping (ORM) แบบ async และใช้ Alembic ในการจัดการการเปลี่ยนแปลงโครงสร้างฐานข้อมูล (migration) อย่างเป็นระบบ



รูปที่ 2.3 PostgreSQL

2.4 Next.js, TypeScript และ Tailwind CSS

Next.js เป็นเฟรมเวิร์กที่พัฒนาต่อยอดจาก React สำหรับสร้างเว็บแอปพลิเคชันที่รองรับทั้ง server-side rendering และ static generation ใช้ร่วมกับ TypeScript เพื่อเพิ่มความปลอดภัยด้านชนิดข้อมูล และ Tailwind CSS ซึ่งเป็น Utility-first CSS Framework สำหรับออกแบบหน้าตาให้ทันสมัยและรองรับการแสดงผลหลายขนาดหน้าจอ (responsive) ในระบบนี้ใช้สร้างหน้าจอส่วนจัดการสำหรับผู้ดูแล (Admin UI)



รูปที่ 2.4 Next.js

2.5 Redis และ Arq

Redis เป็นฐานข้อมูลแบบ in-memory ที่ทำงานได้รวดเร็ว ใช้สำหรับเก็บข้อมูลชั่วคราว (cache) และทำหน้าที่เป็นตัวกลางรับส่งข้อความ (message broker) ส่วน Arq เป็นตัวจัดคิวงาน (task

queue) แบบ async ที่ทำงานร่วมกับ Redis ใช้รันงานเบื้องหลัง เช่น การหมดอายุของประกาศฉุกเฉินอัตโนมัติ การประมวลผลไฟล์สื่อ และคิวคำสั่งควบคุมทีวี



รูปที่ 2.5 Redis

2.6 MinIO

MinIO เป็นระบบจัดเก็บไฟล์แบบ object storage ที่เข้ากันได้กับ Amazon S3 API สามารถติดตั้งใช้งานภายในองค์กรเองได้ ในระบบนี้ใช้ MinIO สำหรับจัดเก็บไฟล์สื่อ เช่น รูปภาพและวิดีโอ ที่ผู้ใช้อัปโหลดเข้ามา



รูปที่ 2.6 MinIO

2.7 Docker และการทำคอนเทนเนอร์ (Containerization)

Docker เป็นเทคโนโลยีที่ช่วยบรรจุแอปพลิเคชันพร้อมสภาพแวดล้อมการทำงานไว้ในคอนเทนเนอร์ ทำให้ติดตั้งและทำงานได้เหมือนกันในทุกเครื่อง ระบบนี้ใช้ Docker Compose ในการจัดการบริการทั้งหมด (backend, worker, ฐานข้อมูล, Redis, MinIO, Admin UI) ให้ทำงานร่วมกัน ทำให้ติดตั้งระบบทั้งหมดได้ด้วยคำสั่งเดียว



รูปที่ 2.7 Docker

2.8 HDMI-CEC และ RS-232

HDMI-CEC (Consumer Electronics Control) เป็นโปรโตคอลที่ส่งคำสั่งควบคุมอุปกรณ์ผ่านสาย HDMI เช่น การสั่งเปิด-ปิดทีวี ส่วน RS-232 เป็นมาตรฐานการสื่อสารแบบ serial ที่ทีวีระดับองค์กรหลายยี่ห้อรองรับสำหรับการควบคุมจากระยะไกล ระบบนี้ใช้ทั้งสองโปรโตคอลในการสั่งเปิด/ปิด/เปลี่ยนช่องสัญญาณเข้าของทีวีหลายยี่ห้อ (เช่น Samsung MDC, LG, Sony)



รูปที่ 2.8 HDMI-CEC และ RS-232

2.9 ความปลอดภัยของระบบ (Security)

ระบบให้ความสำคัญกับความปลอดภัยหลายระดับ ได้แก่ การยืนยันตัวตนด้วย JWT (JSON Web Token) ร่วมกับระบบสิทธิ์ตามบทบาท (Role-Based Access Control: RBAC), การสแกนไวรัสไฟล์ที่อัปโหลดด้วย ClamAV, และการป้องกันการโจมตีแบบ SSRF (Server-Side Request Forgery) ด้วยการตรวจสอบ URL ที่ผู้ใช้ส่งเข้ามาไม่ให้ชี้เข้าสู่เครือข่ายภายใน



รูปที่ 2.9 ความปลอดภัยของระบบ

บทที่ 3
การวิเคราะห์และออกแบบระบบ

ในบทนี้กล่าวถึงการวิเคราะห์ความต้องการของระบบ เครื่องมือที่ใช้ และการออกแบบระบบในด้านสถาปัตยกรรม ฐานข้อมูล และส่วนติดต่อโปรแกรม (API)

3.1 การวิเคราะห์ความต้องการของระบบ

ความต้องการเชิงหน้าที่ (Functional Requirements):

- จัดการเนื้อหาสื่อ (อัปโหลด/อนุมัติ/จัดหมวด)
- จัดผังการแสดงผลแบบหลายโซน (presentation/layout)
- กำหนดตารางการแสดงผลตามช่วงเวลาและกลุ่มจอ
- ควบคุมจอโทรทัศน์ระยะไกล (เปิด/ปิด/เปลี่ยนสัญญาณ)
- ประกาศเหตุฉุกเฉินที่แทรกตารางปกติได้ทันที
- จัดการผู้ใช้และสิทธิ์การเข้าถึง (RBAC)
- เฝ้าระวังสถานะอุปกรณ์และบันทึก audit log

ความต้องการที่ไม่ใช่เชิงหน้าที่ (Non-functional Requirements):

- ความปลอดภัย (การยืนยันตัวตน สิทธิ์ และการตรวจสอบย้อนหลัง)
- ความทนทานต่อเครือข่ายขัดข้อง (offline-friendly ผ่าน manifest)
- ความง่ายในการติดตั้งและดูแล (Docker)
- ใช้เทคโนโลยี Open Source

ระยะเวลาดำเนินงานแสดงด้วยแผนภูมิ Gantt *(รูปที่ 3.1) — [ ] ต้องจัดทำรูป Gantt*

3.2 เครื่องมือที่ใช้ในการพัฒนา

3.2.1 ฮาร์ดแวร์ (Hardware)

- เครื่องคอมพิวเตอร์สำหรับพัฒนา (Developer PC)
- อุปกรณ์ทดสอบการแสดงผล: Android TV, แท็บเล็ต, Raspberry Pi, เครื่อง Windows

3.2.2 ซอฟต์แวร์ (Software)

- ภาษา/เฟรมเวิร์ก: Python 3.12 + FastAPI, Node.js + Next.js + TypeScript
- ฐานข้อมูลและบริการ: PostgreSQL 16, Redis 7, MinIO
- เครื่องมือพัฒนา: Visual Studio Code, Git/GitHub, Docker
- *(รายละเอียดครบทุกเครื่องมือดูที่* `tools-list.md`*)*

3.3 การออกแบบระบบ

3.3.1 สถาปัตยกรรมระบบ (System Architecture)

ระบบประกอบด้วยส่วนหลัก 6 ส่วนที่ทำงานร่วมกันใน Docker stack ได้แก่ (1) Backend (FastAPI) ให้บริการ REST API และตรรกะทางธุรกิจ (2) Worker (Arq) สำหรับงานเบื้องหลัง (3) PostgreSQL ฐานข้อมูลหลัก (4) Redis สำหรับ cache และคิวงาน (5) MinIO สำหรับจัดเก็บไฟล์สื่อ และ (6) Admin UI (Next.js) สำหรับผู้ดูแล โดยส่วนแสดงผล (Player) จะดึงไฟล์ manifest จาก backend แล้วเล่นเนื้อหาแบบ offline-friendly *(รูปที่ 3.2)*

3.3.2 การออกแบบฐานข้อมูล (ER Diagram)

ฐานข้อมูลประกอบด้วยตารางประมาณ 13 ตาราง (entity 12 ตาราง + ตารางความสัมพันธ์แบบ M:N) แบ่งเป็น 5 กลุ่ม ได้แก่ Identity & Audit, Devices, Content & Composition, Playlists & Schedules และ Emergency *(รูปที่ 3.3 — มีไฟล์ ER diagram แล้ว)*

3.3.3 แนวคิดลำดับชั้นเนื้อหาแบบ 5 ระดับ (5-Tier Content Hierarchy)

ระบบออกแบบลำดับชั้นเนื้อหาเป็น 5 ระดับ คือ Content (สื่อดิบ) ⟶ Zone (โซนแสดงผล) ⟶ Presentation (ผังการจัดวาง) ⟶ Plan (ลำดับการเล่น) ⟶ Schedule (เวลาและกลุ่มจอ) แนวคิดนี้ทำให้สามารถนำเนื้อหากลับมาใช้ซ้ำได้ และการแก้ไขตารางครั้งเดียวมีผลกับทั้งกลุ่มจอ

3.3.4 การออกแบบส่วนติดต่อโปรแกรม (API)

API เวอร์ชัน v1 มีทั้งหมดประมาณ 66 endpoints จัดเป็น 13 กลุ่ม (auth, users, devices, groups, contents, presentations, plans, schedules, player, tv, emergency, led, reports) ออกแบบตามแนวทาง REST ยืนยันตัวตนด้วย JWT และมีเอกสารอัตโนมัติแบบ OpenAPI (Swagger)

3.4 วิธีการพัฒนา

พัฒนาด้วยกระบวนการแบบวนซ้ำเป็นเฟส (iterative) ใช้ Git/GitHub ในการจัดการเวอร์ชันโค้ด มีระบบ Continuous Integration (GitHub Actions) สำหรับรันการทดสอบและตรวจ

สอบความปลอดภัยอัตโนมัติ และใช้ Docker เป็นสภาพแวดล้อมการพัฒนาที่สอดคล้องกับ production

3.5 การออกแบบการทดสอบ

ระบบใช้ pytest ร่วมกับ pytest-asyncio และ pytest-cov มีกรณีทดสอบประมาณ 133 กรณี ใน 15 ไฟล์ ครอบคลุมการยืนยันตัวตน อุปกรณ์ กระบวนการจัดการเนื้อหา การควบคุมทีวี การประกาศฉุกเฉินและ audit, มิดเดิลแวร์ความปลอดภัย, รายงาน (reports), การควบคุม LED, การป้องกัน SSRF และงานเบื้องหลัง นอกจากนี้ยังมีการทดสอบแบบ End-to-End (smoke test) และระบบ CI ที่รันการทดสอบอัตโนมัติทุกครั้งที่มีการแก้ไขโค้ด

บทที่ 4

ผลการดำเนินงาน

ในบทนี้นำเสนอผลการพัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล ทั้งในส่วนความสามารถของแต่ละโมดูล หน้าจอการใช้งานจริง และส่วนแสดงผล

4.1 ภาพรวมระบบที่พัฒนา

ระบบได้รับการพัฒนาจนสามารถใช้งานได้ครบวงจร ประกอบด้วยส่วน backend (REST API), ส่วนจัดการสำหรับผู้ดูแล (Admin UI) และส่วนแสดงผล (Player) ทั้งบนเว็บและ Android รองรับกระบวนการทำงานตั้งแต่การอัปโหลดเนื้อหา ⟶ จัดผังการแสดงผล ⟶ ตั้งตารางเวลา ⟶ จอแสดงผลจริง ⟶ การประกาศเหตุฉุกเฉิน

4.2 โมดูลและความสามารถหลัก

โมดูล	**ความสามารถ**
Auth & Users	ยืนยันตัวตนด้วย JWT + RBAC 7 บทบาท, ล็อกบัญชีเมื่อล็อกอินผิดซ้ำ, รองรับ cookie auth
Devices	จับคู่อุปกรณ์ด้วยรหัส, heartbeat, telemetry (CPU/อุณหภูมิ/uptime), จัดกลุ่มจอแบบ M:N
Contents	อัปโหลดสื่อ + ตรวจซ้ำด้วย sha256 + สร้าง thumbnail + กระบวนการอนุมัติ (draft⟶pending⟶approved/rejected)
Presentations	จัดผัง layout หลายโซน (composer) ใน Admin UI
Plans	จัดลำดับ presentation พร้อมกำหนดระยะเวลาแสดงผล
Schedules	ผูก plan กับกลุ่มจอ + กฎเวลาแบบ RRULE (iCal) + ช่วงเวลารายวัน + ลำดับความสำคัญ + การอนุมัติ
TV Control	สั่งเปิด/ปิด/เปลี่ยนสัญญาณผ่าน HDMI-CEC และ RS-232 หลายยี่ห้อ ผ่านคิวใน Redis
LED Control	ควบคุมจอ LED ผ่านโมดูลควบคุมเฉพาะ *(ฟีเจอร์เพิ่มเติม)*
Emergency	ประกาศฉุกเฉิน override ทุกตาราง + หมดอายุอัตโนมัติ + ต้องการการยืนยันรับทราบ (ack)
Reporting	บันทึกประวัติการเล่น (playback_logs) + สรุปรายงานและส่งออกข้อมูล *(ฟีเจอร์เพิ่มเติม)*
Monitoring	Prometheus metrics + readiness/liveness + audit log + Sentry + การสำรองข้อมูล

โมดูล	ความสามารถ
Player	แสดงผลหลายโซน + วนเล่นตาม plan + overlay ประกาศฉุกเฉิน + cache แบบ offline

4.3 หน้าจอส่วนจัดการสำหรับผู้ดูแล (Admin UI)

ส่วนจัดการประกอบด้วยหน้าจอหลักหลายหน้า แสดงผลตัวอย่างดังนี้

- ☐ รูปที่ 4.1 หน้าเข้าสู่ระบบ (Login)
- ☐ รูปที่ 4.2 หน้าแดชบอร์ด (Dashboard)
- ☐ รูปที่ 4.3 หน้าจัดการเนื้อหา (Contents)
- ☐ รูปที่ 4.4 หน้าจัดผังการแสดงผล (Presentations / Composer)
- ☐ รูปที่ 4.5 หน้าจัดลำดับ (Plans)
- ☐ รูปที่ 4.6 หน้าจัดตารางเวลา (Schedules)
- ☐ รูปที่ 4.7 หน้าจัดการอุปกรณ์และกลุ่มจอ (Devices)
- ☐ รูปที่ 4.8 หน้าควบคุมทีวี/LED (TV/LED Control)
- ☐ รูปที่ 4.9 หน้าประกาศฉุกเฉิน (Emergency)
- ☐ รูปที่ 4.10 หน้ารายงานและการเฝ้าระวัง (Reports / Monitoring)

4.4 ส่วนแสดงผล (Player)

ส่วนแสดงผลทำหน้าที่ดึง manifest จาก backend แล้วแสดงเนื้อหาตามผังและตารางที่กำหนด รองรับการแสดงหลายโซนพร้อมกัน การวนเล่นตามลำดับ และการแสดง overlay เมื่อมีประกาศฉุกเฉิน รวมถึงการ cache เนื้อหาเพื่อให้เล่นต่อได้แม้เครือข่ายขัดข้อง

- ☐ รูปที่ 4.11 ตัวอย่างการแสดงผลบนจอ (Player – normal)
- ☐ รูปที่ 4.12 ตัวอย่างการแสดง overlay ประกาศฉุกเฉิน (Player – emergency)

บทที่ 5
สรุปผลและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

โครงงานนี้ได้พัฒนาระบบจัดการจอประชาสัมพันธ์ดิจิทัล (Digital Signage Management System) จนสามารถใช้งานได้ครบวงจร ตั้งแต่การจัดการเนื้อหา การจัดผังการแสดงผล การกำหนดตารางเวลา การควบคุมจอโทรทัศน์และจอ LED ระยะไกล ไปจนถึงการประกาศเหตุฉุกเฉินและการเฝ้าระวังสถานะอุปกรณ์ ระบบทำงานบนสถาปัตยกรรม 6 ส่วนในรูปแบบคอนเทนเนอร์ มีส่วนจัดการสำหรับผู้ดูแล (Admin UI) และส่วนแสดงผล (Player) ทั้งบนเว็บและ Android ที่พร้อมใช้งานจริง โดยบรรลุวัตถุประสงค์ทั้งหมดที่ตั้งไว้ในบทที่ 1

ในด้านคุณภาพ ระบบมีกรณีทดสอบอัตโนมัติประมาณ 133 กรณี ครอบคลุมส่วนสำคัญของระบบ และมีระบบ CI ตรวจสอบความถูกต้องและความปลอดภัยทุกครั้งที่แก้ไขโค้ด

5.2 ปัญหาที่พบและแนวทางแก้ไข

ปัญหาที่พบ	แนวทางแก้ไข
เครือข่ายภายในอาจขัดข้องชั่วคราว ทำให้จอหยุดแสดงผล	ออกแบบให้ player ดึง manifest แล้ว cache เนื้อหาไว้เล่นต่อได้แบบ offline-friendly
ทีวีมีหลายยี่ห้อ คำสั่งควบคุมต่างกัน	ออกแบบชั้นควบคุมแบบ enum protocol รองรับหลายยี่ห้อผ่าน HDMI-CEC และ RS-232
ความเสี่ยงด้านความปลอดภัยจากไฟล์อัปโหลดและ URL ภายนอก	เพิ่มการสแกนไวรัสด้วย ClamAV และการป้องกัน SSRF ในการตรวจสอบ URL
ต้องควบคุมสิทธิ์การเข้าถึงและตรวจสอบย้อนหลังได้	ใช้ระบบ RBAC 7 บทบาท ร่วมกับกระบวนการอนุมัติและการบันทึก audit log

5.3 ข้อเสนอแนะและการพัฒนาต่อยอด

แนวทางการพัฒนาต่อยอดในอนาคต (Phase 7) ได้แก่

1. รองรับ HDMI-CEC แบบ native บนอุปกรณ์แสดงผลให้สมบูรณ์ยิ่งขึ้น
2. เสริมความปลอดภัยของการยืนยันตัวตนด้วย httpOnly cookie
3. รองรับหลายภาษา (i18n) ในส่วนจัดการ
4. ขยายระบบ CI/CD ให้ครอบคลุมการ deploy อัตโนมัติ

5. เพิ่มตารางข้อมูลแบบ time-series สำหรับ telemetry เพื่อวิเคราะห์แนวโน้มของอุปกรณ์

5.4 สิ่งที่ได้เรียนรู้จากการฝึกงาน

(เขียนจากประสบการณ์จริงของผู้จัดทำ — เช่น การออกแบบระบบจากความต้องการจริง การทำงานกับเทคโนโลยีหลายส่วนร่วมกัน การเขียนการทดสอบอัตโนมัติ การใช้ Docker และ Git ในการพัฒนาเป็นทีม ฯลฯ) — [ ] เติมเนื้อหาส่วนตัว